Application System Brief

Repo-backed product and architecture summary of the current application implementation. This brief is based on the codebase snapshot available on March 26, 2026.

Product position: AI data architect + governed report builder + scheduled narrative reporting system.

Primary promise: define an analysis once in chat, govern the data safely, generate a business-consumable report, and optionally run that report again on a deterministic schedule.

Core rule set: read-only data access, SELECT-only SQL, evidence row cap of 200 rows, deterministic cron scheduling, and customer-facing outputs that stay structured and comparable over time.

1. What the Application Is

The application is an analytics product built around contracts rather than dashboards. A user connects a source, asks for a report in natural language, works through scope clarifications, runs governed data preparation and analysis, and receives a narrative HTML/PDF report. That run can then be used for report Q&A, business case analysis, and scheduled reruns.

The product is designed to behave like an analyst and reporting system layered on top of governed warehouse access. It is not meant to write back to customer data, replace warehouse compute, or bypass governance.

2. Current User-Facing Surfaces

Surface	Route	Purpose
Login	`/login`	Workspace authentication.
Chat	`/`	Main operating surface for single query, scoped multi-query analysis, report generation, clarification, business case work, and scheduled-report follow-up.
Connect	`/connect`	Guided source onboarding, governance setup, cataloging, activation, and edit/disconnect management.
Connection Guide	`/connect/guide`	HTML help guide for connecting supported databases and handling TLS issues.
Usage Metrics	`/usage`	Operational query-log and governed usage view.
Global Config	`/config`	Business context and metric-definition management used by planning and reporting agents.
Scheduled Reports	`/scheduled`	View saved scheduled report profiles, inspect run history, pause/activate schedules, and reopen a scheduled run in chat.

3. High-Level Architecture

Web app

The web app is the main operator surface. It renders chat, connection setup, config, usage metrics, and scheduled reports. It also proxies many API calls so the browser talks to the web layer instead of the API directly for most session and product flows.

API app

The API owns contract creation, run preparation, run execution, scheduling endpoints, business case and report clarification routes, report artifact serving, and metadata persistence.

Worker

Shared packages

The worker refreshes locked report schedules, computes deterministic due jobs from cron + timezone, queues them, dispatches scheduled or retry runs, and records retry/dead-letter events.

Shared packages define schemas, contracts, scheduling types, orchestration state, guardrails, report rendering, SQL policy checks, and LLM client interfaces.

> **Core domain objects:** `ReportContract`, `ReportRun`, `ScheduledReportProfile`, semantic entities/fields/relationships/metrics/dimensions, and persisted chat sessions.

4. End-to-End Operating Flow

1. User connects a governed data source through the Connect flow.
2. User optionally sets shared business context and metric definitions in Global Config.
3. User starts a chat and asks for either a single analysis or a multi-question report.
4. The conversation/orchestration layer scopes questions, asks clarifications, and suggests optional related questions when appropriate.
5. When scope is ready, the user runs data preparation. Prepared queries can be reviewed, copied, and edited before final analysis.
6. The report contract is prepared and then executed through the pipeline.
7. The product generates an HTML report, offers PDF download, and unlocks report clarification and business case follow-ups.
8. The user can optionally schedule the report, which stores cadence, question rerun logic, query templates, and the HTML template snapshot.
9. The worker reruns the locked contract on schedule and injects change notes versus the last comparable run.

5. AI Agents and Deterministic Tools

Component	Role	Current status
Agent A - Contract Builder	Turns user intent into a report contract.	Documented in agent map.
Agent B - Semantic Mapper	Maps user language to governed semantic objects.	Documented in agent map.
Agent C - Planner / Data Architect	Breaks work into plan steps and query strategy.	Active.
Tool T1 - SQL Policy Engine	SELECT-only enforcement, allowlist/schema enforcement, limit insertion.	Deterministic guardrail.
Tool T2 - Data Plane Executor	Runs governed SQL against the connected source.	Deterministic runtime.
Agent D - Evidence Reducer & Batch Controller	Controls batch evidence handling and reduction.	Active in pipeline.
Agent E - Batch Analyst	Answers scoped questions from prepared evidence.	Active.
Batch Forecast Agent	Forecast-specialized batch analysis using the same model family as batch analyst.	Active.
Agent F - Aggregator	Builds concise business summary structure across analyses.	Active.
Agent G - Renderer / Composer	Generates customer-facing HTML report sections.	Active.
Agent H - QA / Judge	Quality and consistency checks for report output.	Active in overall pipeline design.
Agent I - Data Preparation	Prepares safe, reduced evidence before analysis.	Active.
Agent J - Payload QA	Answers run-payload questions from a generated run.	Active.

Component	Role	Current status
Report Clarification Agent	Answers questions grounded in report HTML, exec brief, analysis payloads, metric definitions, and business context.	Active.
Batch Business Case Agent	Builds a detailed business case for selected actionable recommendations and can request more evidence.	Active.
Conversation Orchestrator	Controls chat state, route selection, scope clarification, button-driven transitions, and session naming.	Active.
Query Router	Chooses workflow path and helps interpret scoping turns.	Active.

6. Current Model Stack

The default runtime provider is OpenRouter. The current repo defaults are:

Area	Current model	Notes
Conversation orchestration	`openai/gpt-5.4`	Main chat/orchestration surface.
Batch analyst	`openai/gpt-5.4`	Deep analysis model.
Forecast analyst	`openai/gpt-5.4`	Explicitly shares analyst model selection.
Report composer	`openai/gpt-5.4`	HTML report generation.
Report clarification	`openai/gpt-5.4`	Answers grounded questions on generated reports.
Business case	`openai/gpt-5.4`	Produces business case outputs and clarification prompts.
Query strategist	`anthropic/claude-opus-4.6`	Query planning and support-query generation.
Data preparation	`anthropic/claude-opus-4.6`	Preparation-oriented planning.
Query router / single-query helpers	`anthropic/claude-opus-4.6`	Front-door routing and scoped interpretation.
Embeddings	`openai/text-embedding-3-small`	Chat retrieval memory.
Rerank	`openai/gpt-4.1-mini`	RAG support.

7. Database Connectivity and Governance

The product currently supports four user-facing source types in the Connect UI: Postgres, MySQL, Snowflake, and BigQuery. At runtime, the connection manager also recognizes Supabase and Neon as Postgres-compatible providers.

Connection wizard

1. **Source:** choose provider, then enter provider-specific connection details.
2. **Governance:** validate tables/columns and save the allowlist.
3. **Cataloging:** the UI shows a cataloging/loading step and table/column counts.
4. **Activate:** run a safe sample query and submit the source for governed use.

Edit mode

The connection UI also supports an edit/manage path for existing sources. Users can inspect connected-source details, review allowlisted objects, change governance, and disconnect.

Guardrails on data access

- Write access is explicitly denied. The system enforces `deny_write: true`.
- SQL must remain SELECT-only.

- Queries are checked against allowlisted relations and schemas.
- A row cap is applied in the governed execution path.
- Query logs and audit events are persisted for usage and operational review.

8. Semantic, Config, and Context Layers

The product has an explicit semantic store for:

- entities
- fields
- relationships
- metrics
- dimensions

In addition, the Global Config surface lets the team maintain shared business context and metric definitions. These definitions are carried into scoping, query strategy, analyst interpretation, report composition, and follow-up agents.

9. Chat and Analysis Flows

Single-query flow

The user asks one analytical question. The router selects the appropriate direct analysis path and returns a governed answer.

Multi-query scoped flow

The user describes a report, the chat breaks it into atomic questions, asks clarifications, supports add/remove/include-suggestion edits, then unlocks data preparation and final analysis.

Data preparation step

Before final analysis, the user can run data preparation. This produces prepared payloads, validation notes, sample rows, and query previews. Each prepared query can be copied or edited before final analysis, and those overrides are persisted into the contract as `prepared_query_overrides`.

Post-run actions

After a report is generated, the chat exposes:

- Ask clarifications on the report
- Ask for business case analysis
- Schedule this report

10. Report Generation Pipeline

1. Load the contract and effective context.
2. Resolve scoped questions, clarifications, and any saved overrides.
3. Generate or refine strategy queries.
4. Run SQL guardrails and governed execution.
5. Reduce evidence to at most 200 rows.
6. Produce analyst outputs per question.
7. Build summaries, KPI checks, and change notes versus the previous comparable run.
8. Compose HTML report output.
9. Serve HTML directly and render downloadable PDF from that HTML.
10. Persist artifacts and audit events.

The HTML composer is instructed to reuse a previous HTML template when available for scheduled reruns. It is also told to add a concise “What changed since last run” subsection when change-summary notes are present.

11. Report Outputs and Follow-Ups

HTML and PDF

The canonical customer-facing report artifact is HTML. The product also provides a PDF download endpoint by rendering that HTML to PDF. If PDF generation fails at runtime, the API can fall back to downloadable HTML.

Report clarification flow

The report clarification agent answers grounded follow-up questions using the report HTML, exec brief, per-question summaries, analysis payloads, prepared payloads, metric definitions, and business context.

Business case flow

The business case flow first extracts only genuinely actionable recommendations from a report run. If nothing in the report qualifies as a business-case-worthy recommendation, the system returns no candidates. If a candidate exists, the business case agent can ask for assumption clarifications, optionally request more governed evidence through the query strategist/data plane path, and then produce a structured business case.

12. Scheduled Reports

A completed run can be turned into a scheduled report profile. The scheduler stores more than just cadence. It stores the intended future behavior of each question and enough state to make reruns stable.

Stored with schedule	Why it matters
Frequency, timezone, cron	Deterministic recurrence.
Windowing instructions	Explains how date windows should roll forward on future runs.
Additional instructions	Captures any special operator notes for future reruns.
Question execution plan	Preserves how each scoped question should behave on future runs.
Query template snapshot	Carries forward the source-run query logic for reuse.
Report template HTML snapshot	Lets the composer reuse the prior report structure and style.

The Scheduled Reports page presents report profiles as tiles, lets users inspect detail and run history, reopen a completed run in chat, and pause or reactivate the schedule. Paused schedules clear the active cron on the contract until they are reactivated.

13. Worker Runtime and Deterministic Scheduling

The worker uses a deterministic scheduler. It parses 5-field cron, validates timezone names, computes the next eligible run minute-by-minute, and registers only locked contracts with an active schedule. On each tick it:

1. refreshes schedules from the API
2. collects due jobs
3. enqueues them
4. dispatches runs through the API
5. applies exponential retry backoff
6. moves permanently failing jobs to a dead-letter list

14. Persistence and Storage

The metadata layer stores:

- semantic records
- report contracts and contract versions
- report runs
- scheduled report profiles
- platform users
- chat sessions and their message history
- RAG chunks for chat retrieval
- audit logs

The repo includes both a Postgres-backed metadata store and an in-memory implementation for tests and local runtime scenarios.

15. Current Web-to-API Boundary

The web app acts as a thin product-facing layer in front of the API for most browser-used flows, including chat sessions, scheduling, database/governance routes, and config. Some report artifact routes, such as run status, HTML, and PDF fetches, use direct API-client handling instead of the generic proxy helper.

16. Supported Operational Features Today

- Authentication and persisted chat history
- Single-query and scoped multi-query analysis
- Governed source connection and allowlisting
- Business context and metric-definition management
- Prepared query preview, copy, and edit before final analysis
- HTML report generation and PDF download
- Grounded report clarification Q&A
- Business case generation for actionable recommendations
- Scheduled report creation, inspection, pause/reactivate, and rerun support
- Usage metrics and auditability of governed SQL

17. Core Constraints and Guardrails

- No customer-database write access.
- SELECT-only SQL enforcement.
- Evidence row cap of 200 rows.
- Deterministic scheduling with cron + timezone.
- Business-consumable output structure with consistent sections.
- Change-oriented reporting against prior comparable runs when available.

18. Local Development and Validation

Repo-documented local setup includes:

- `pnpm install`
- `pnpm approve-builds`
- `docker compose -f infra/docker-compose.yml up -d`
- `pnpm db:migrate`
- `pnpm dev`
- `pnpm test`

The standard runtime split is web on port 3000, API on port 4000, with Postgres and Redis provided through the local Docker composition.

19. Summary of What Exists Today

The application already has the shape of a real governed analytics product rather than a prototype chatbot. The main pieces are present: governed data connection, scoped chat analysis, data preparation, report generation, follow-up report Q&A, business case analysis, deterministic scheduling, rerun reuse, and operational storage around contracts, runs, sessions, and audit history.

The most important product idea visible in the repo today is that the system is built around reusable report contracts and scheduled reruns rather than ad hoc isolated answers. The codebase already reflects that direction in the contract schema, run pipeline, schedule profiles, and worker runtime.

Generated from the current repo snapshot. Source references used for this brief include `README.md`, `PRD.md`, `ARCHITECTURE.md`, `docs/AGENT_MAP.md`, web/app routes, API routes, scheduling types, worker runtime, and current environment defaults in `.env.example`.